

Moving Average and Flop Count Problem

Your task is to compute a moving average of an integer *list* where a window size for computing the moving average k is given. The formulation for moving average is as follows:

for $i = 1, 2, \dots, n$

$$o_i = \begin{cases} \frac{1}{k} \sum_{j \in (i - \lfloor \frac{k}{2} \rfloor : i + \lfloor \frac{k}{2} \rfloor)} a_j, & \text{if } \lfloor \frac{k}{2} \rfloor \leq i \leq n - \lfloor \frac{k}{2} \rfloor \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

Input Format

- A list of n integers $\mathbf{a} = [a_1, a_2, \dots, a_n]$; and
- An odd number k for the window size.

Output Format

- A list of n floats (e.g., `output = [o1, o2, ..., on]`) which is the result of the moving average operation;
- An integer `flop` which is the flop count of the moving average operation.

Constraints

- $3 \leq n \leq 100$;
- k is an odd number and $k \leq n - 2$; and
- $1 \leq a_i \leq 100$ for all i from 1 to n .

Sample Input

```
a = [1, 2, 3, 4, 5, 6]
k = 3
```

Sample Output

```
[0, 2.0, 3.0, 4.0, 5.0, 0], 12
```

Implementation

Goal: Fill in the following function:

```
def moving_average(list_a: list, k:int):
    ...
    return output, flop # Return the resulting list and the number of flop counts.
```

Warnings

- You are **NOT ALLOWED** to use the Numpy package.